

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”



Технології графічного процесінгу

Прискорені обчислення для Python з Numba

Практична робота #4

Виконав:
Олександр Ковальов
Група:
ІМ-51мн
Курс:
1

1 травня 2026 р.

1 Мета

Навчитися використовувати Numba для компіляції та прискорення Python-функцій на CPU і GPU, писати власні векторизовані функції з виконанням на GPU, а також керувати передачею даних між CPU і GPU, щоб уникнути зайвих накладних витрат.

2 Завдання

Відкрийте стартовий код завдання за посиланням: colab.research.google.com/github/YKochura/ac-kpi/blob/main/labs/pr4_python_numba.ipynb

Реалізуйте три версії функції `create_hidden_layer`: на чистому NumPy (стартовий код завдання), з Numba CPU (`@vectorize`) та з Numba GPU (`@vectorize(target='cuda')`). Для кожної реалізації виміряйте середній час виконання (рекомендується попередньо виконати JIT-прогрівання). Numba при першому виклику функції компілює Python-код у машинний код під конкретний тип даних – це займає десятки мілісекунд і не відображає реальну швидкість обчислень. Усі наступні виклики вже використовують скомпільований код і працюють швидше.

В GPU-версії дані між CPU і GPU передавайте лише один раз, а не після кожної операції.

Результати оформіть у вигляді протоколу: таблицю замірів для різної кількості нейронів $n = 1\text{M}$ і $n = 100\text{M}$, порівняння продуктивності та висновки щодо отриманого прискорення і ролі оптимізації передачі даних.

Протокол виконання лабораторної роботи

Таблиця замірів продуктивності

Табл. 1: Результати вимірювання часу виконання функції `create_hidden_layer`

Версія реалізації	Час виконання ($n = 10^6$)	Час виконання ($n = 10^8$)
Чистий NumPy	9.74 ms	1.87 s
Numba CPU	8.89 ms	970 ms
Numba GPU	3.88 ms	295 ms

Порівняння продуктивності

Аналіз отриманих результатів вимірювання часу виконання яскраво демонструє переваги використання JIT-компіляції та апаратного прискорення. Базова реалізація на чистому NumPy показала найнижчу продуктивність під час роботи з великим масивом даних розміром у сто мільйонів елементів, витративши 1.87 секунди. Це зумовлено тим, що під час виконання векторизованих математичних операцій створюються нові тимчасові масиви в оперативній пам'яті для кожного проміжного кроку. Використання JIT-компілятора Numba для центрального процесора дозволило скоротити час виконання до 970 мілісекунд, забезпечивши прискорення майже вдвічі. Декоратор `vectorize` зливає всі дрібні операції в один оптимізований цикл на рівні машинного коду, усуваючи необхідність створення проміжних масивів. Реалізація з використанням Numba CUDA для графічного процесора продемонструвала найвищу ефективність, виконавши завдання за 295 мілісекунд, що у понад шість разів швидше за базовий підхід. На меншому обсязі даних в один мільйон елементів різниця між процесорними підходами мінімальна через накладні витрати на виклик функцій, проте GPU все одно зберігає лідерство із результатом 3.88 мілісекунди.

Висновки щодо отриманого прискорення та ролі оптимізації передачі даних

Застосування графічного процесора для векторизованих обчислень забезпечує суттєвий приріст продуктивності завдяки масивній паралелізації, що особливо помітно на великих обсягах даних. Вирішальну

роль у досягненні такого високого показника відіграла оптимізація передачі даних між оперативною пам'яттю хоста та відеопам'яттю пристрою. Пропускна здатність шини PCIe є фізичним обмеженням, яке може повністю нівелювати обчислювальну потужність відеокарти при нераціональному використанні. Одноразове завантаження початкових масивів у пам'ять відеокарти перед початком обчислень, виконання всіх математичних перетворень локально на пристрої та одноразове повернення фінального результату дозволили уникнути критичних накладних витрат. Отже, ефективне управління пам'яттю є настільки ж важливим етапом у технологіях графічного процесінгу, як і безпосереднє розпаралелювання обчислень.

3 Лістинг.

Лістинг 1: Код.

```

import numpy as np
import math
from numba import vectorize, cuda

\# Set initial number of neurons for the first test
n = 1\_\_000\_\_000

\# Generate initial data
greyscales = np.floor(np.random.uniform(0, 255, n).astype(np.float32))
weights = np.random.normal(.5, .1, n).astype(np.float32)

\# Pack arguments for convenience
arguments = \{
    'greyscales': greyscales,
    'weights': weights
\}

\# Pure NumPy implementation

def normalize\_np(grayscales):
    return greyscales / 255.0

def weigh\_np(values, weights):
    return values * weights

def activate\_np(values):
    return (np.exp(values) - np.exp(-values)) / (np.exp(values) + np.exp(-values))

def create\_hidden\_layer\_numpy(greyscales, weights):
    normalized = normalize\_np(greyscales)
    weighted = weigh\_np(normalized, weights)
    return activate\_np(weighted)

\# Numba CPU implementation

@vectorize(['float32(float32)'], target='cpu')
def normalize\_cpu(grayscales):
    return greyscales / 255.0

@vectorize(['float32(float32, _float32)'], target='cpu')
def weigh\_cpu(values, weights):
    return values * weights

@vectorize(['float32(float32)'], target='cpu')
def activate\_cpu(values):
    \# Use math.exp instead of numpy.exp for Numba compatibility
    return (math.exp(values) - math.exp(-values)) / (math.exp(values) + math.exp(-values))

def create\_hidden\_layer\_cpu(greyscales, weights):
    normalized = normalize\_cpu(greyscales)

```

```

weighted = weigh\_cpu(normalized, weights)
return activate\_cpu(weighted)

\# Numba GPU implementation with optimized data transfer

@vectorize(['float32(float32)'], target='cuda')
def normalize\_gpu(grayscales):
return grayscales / 255.0

@vectorize(['float32(float32, float32)'], target='cuda')
def weigh\_gpu(values, weights):
return values * weights

@vectorize(['float32(float32)'], target='cuda')
def activate\_gpu(values):
return (math.exp(values) - math.exp(-values)) / (math.exp(values) + math.exp(-values))

def create\_hidden\_layer\_gpu(grayscales, weights):
\# Transfer arrays to device memory once to avoid PCIe bottleneck
d\_grayscales = cuda.to\_device(grayscales)
d\_weights = cuda.to\_device(weights)

\# Perform all calculations on the device
d\_normalized = normalize\_gpu(d\_grayscales)
d\_weighted = weigh\_gpu(d\_normalized, d\_weights)
d\_activated = activate\_gpu(d\_weighted)

\# Copy the final result back to host memory
return d\_activated.copy\_to\_host()

\# JIT Warm-up sequence
\# Execute functions with a small subset to trigger compilation

warmup\_g = grayscales[:10]
warmup\_w = weights[:10]

\_ = create\_hidden\_layer\_numpy(warmup\_g, warmup\_w)
\_ = create\_hidden\_layer\_cpu(warmup\_g, warmup\_w)
\_ = create\_hidden\_layer\_gpu(warmup\_g, warmup\_w)

print('Warm-up complete. Functions are compiled.')

\# Benchmark for n = 1M

print('NumPy version (1M):')
%timeit create\_hidden\_layer\_numpy(**arguments)

print('Numba CPU version (1M):')
%timeit create\_hidden\_layer\_cpu(**arguments)

print('Numba GPU version (1M):')
%timeit create\_hidden\_layer\_gpu(**arguments)

\# Generate data and benchmark for n = 100M

```

```
n\_large = 100\_000\_000

greyscales\_large = np.floor(np.random.uniform(0, 255, n\_large).astype(np.float32))
weights\_large = np.random.normal(.5, .1, n\_large).astype(np.float32)

arguments\_large = \{
    ‘greyscales’: greyscales\_large,
    ‘weights’: weights\_large
\}

print(‘NumPy version (100M):’)
\%timeit create\_hidden\_layer\_numpy(**arguments\_large)

print(‘\ textbackslash {}nNumba CPU version (100M):’)
\%timeit create\_hidden\_layer\_cpu(**arguments\_large)

print(‘\ textbackslash {}nNumba GPU version (100M):’)
\%timeit create\_hidden\_layer\_gpu(**arguments\_large)
```